2520030117

OS PRACTICAL – 7

PART A — MEMORY ADDRESS SPACE ANALYSIS

AIM

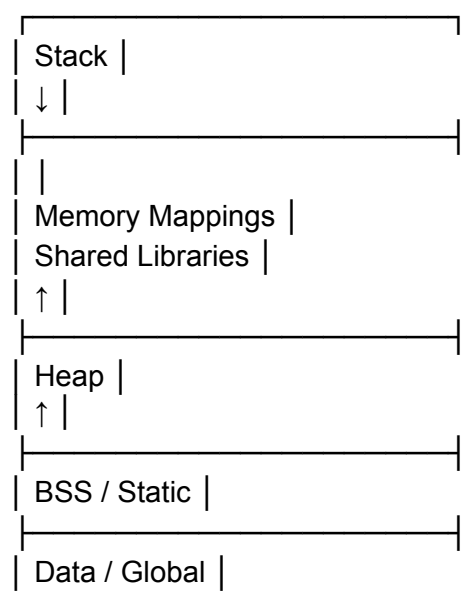
To write a program that prints the addresses of **code, global, static, heap, and stack variables**, analyze the Linux process address space layout, and explain the purpose of each memory segment.

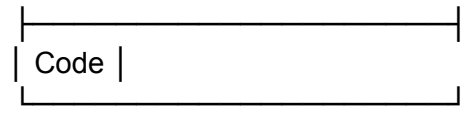
OBJECTIVES

1. To understand the memory layout of a Linux process.
2. To print the address of a code/function variable.
3. To print the address of global and static variables.
4. To print the address of a heap variable.
5. To print the address of a stack variable.
6. To analyze the `/proc/<PID>/maps` file.
7. To understand virtual memory mapping.

MEMORY SEGMENTS

A Linux process generally has memory areas like:





Low Address

Simple meaning

Segment	Contains
Code/Text	Program instructions
Global/Data	Initialized global variables
BSS	Uninitialized global/static variables
Static	Static variables
Heap	Dynamically allocated memory
Stack	Local variables and function calls

PROGRAM

File name

memory_layout.c

COPY-PASTE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;
static int static_var = 200;

void show_code_address()
{
    printf("Code address : %p\n", (void *)show_code_address);
}

int main()
{
    int stack_var = 300;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
```

```
    {
        perror("malloc");
        return 1;
    }

    *heap_var = 400;

    printf("===== MEMORY ADDRESS SPACE =====\n");

    printf("Process ID : %d\n", getpid());

    show_code_address();

    printf("Global variable : %p\n", (void *)&global_var);

    printf("Static variable : %p\n", (void *)&static_var);

    printf("Heap variable : %p\n", (void *)heap_var);

    printf("Stack variable : %p\n", (void *)&stack_var);

    printf("\n===== VARIABLE VALUES =====\n");

    printf("Global value : %d\n", global_var); printf("Static

    value : %d\n", static_var);

    printf("Heap value : %d\n", *heap_var);

    printf("Stack value : %d\n", stack_var);

    printf("\nProcess is running...\n");
    printf("Press Enter to exit.\n");

    getchar();
    free(heap_var);

    return 0;
}
```

PART B — `/proc/<PID>/maps` ANALYSIS

Linux provides information about a running process through the `/proc`

filesystem. For a process with PID `1234`, we can view its memory mappings

using:

cat /proc/1234/maps

This shows the virtual memory regions used by the process.

ALGORITHM

Program

1. Start the program.
2. Create global and static variables.
3. Create a local stack variable.
4. Dynamically allocate memory using `malloc()`.
5. Print the process ID.
6. Print the address of the code.
7. Print the addresses of global and static variables.
8. Print the heap address.
9. Print the stack address.
10. Keep the process running.
11. Analyze its memory map using `/proc/<PID>/maps`.
12. Release heap memory.
13. Terminate the program.

NOW — UBUNTU STEP BY STEP

STEP 1 — Create the folder

Open Ubuntu.

Type:

mkdir os_practical7

Then:

cd os_practical7

You should see:

kruthi@LAPTOP:~/os_practical7$

STEP 2 — Create the C file

Type:

nano memory_layout.c

STEP 3 — Paste the code

Paste the complete `memory_layout.c` code given above.

STEP 4 — Save

Press:

Ctrl + O

Then:

Enter

Then:
Ctrl + X

STEP 5 — Compile

Type:

gcc memory_layout.c -o memory_layout

If there is **no error**, compilation is successful. ✅

STEP 6 — Run

Type:

./memory_layout

You should get something similar to:

===== MEMORY ADDRESS SPACE =====
Process ID : 700

Code address : 0x55xxxxxxxxx
Global variable : 0x55xxxxxxxxx
Static variable : 0x55xxxxxxxxx
Heap variable : 0x55xxxxxxxxx
Stack variable : 0x7fffffffxxx

===== VARIABLE VALUES =====
Global value : 100
Static value : 200
Heap value : 400
Stack value : 300

Process is running...
Press Enter to exit.

⚠️ **Your addresses will be different. That's completely normal.**

⚠️ IMPORTANT

When you see:

Process ID : 700

DO NOT PRESS ENTER YET.

The program must remain running because we need its PID to inspect:

/proc/<PID>/maps

STEP 7 — TAKE SCREENSHOT

📸 Take a screenshot showing:

===== MEMORY ADDRESS SPACE =====
Process ID : XXXXX

Code address : ...
Global variable : ...
Static variable : ...
Heap variable : ...
Stack variable : ...

This is your **Program Output Screenshot**.

STEP 8 — OPEN SECOND TERMINAL

Keep the first terminal running.

Open a **second Ubuntu terminal**.

Go to the folder:

cd ~/os_practical7

STEP 9 — View `/proc/<PID>/maps`

Suppose your PID was:

700

Then type:

cat /proc/700/maps

⚠️ Replace `700` with **your actual PID**.

You should see something similar to:

55a1c0000000-55a1c0001000 r--p ... /home/kruthi/os_practical7/memory_layout
55a1c0001000-55a1c0002000 r-xp ... /home/kruthi/os_practical7/memory_layout
55a1c0002000-55a1c0003000 r--p ... /home/kruthi/os_practical7/memory_layout
55a1c0003000-55a1c0004000 rw-p ... /home/kruthi/os_practical7/memory_layout
55a1d0000000-55a1d0021000 rw-p ... [heap]
7ffdxxxxxxxx-7ffdxxxxxxxx rw-p ... [stack]

STEP 10 — Easier `/proc/maps` command

Instead of showing everything, you can use:

cat /proc/700/maps | grep -E "heap|stack|memory_layout"

Again, replace `700` with your PID.

This makes the important regions easier to identify.

You should see entries containing:

memory_layout
[heap]
[stack]

STEP 11 — TAKE SCREENSHOT

�� Take a screenshot of the `/proc/<PID>/maps` output.

This is your:

Memory Mapping Analysis Screenshot

STEP 12 — Check memory information

You can also run:

pmap 700

Replace `700` with your PID.

If `pmap` is available, it will show the process memory map in a more readable

format. You can also use:

pmap -x 700

STEP 13 — FINISH THE PROGRAM

Go back to the **first terminal**.

You should still see:

Press Enter to exit.

Press:

Enter

The program terminates.

WHAT TO WRITE FOR ANALYSIS

1. Code Segment

The **code segment** contains the machine instructions of the program.

The address printed using:

&show_code_address

belongs to the program's code area.

2. Global Segment

The global variable:

int global_var = 100;

is stored in the program's global/data area.

3. Static Segment

The static variable:

static int static_var = 200;

has static storage duration and is stored in the program's data/BSS-related area depending on initialization.

4. Heap

The heap is used for dynamically allocated memory.

We allocated memory using:

malloc()

The address is stored in:

heap_var
The corresponding region can be seen in:

[heap]

inside `/proc/<PID>/maps`.

5. Stack

The local variable:

int stack_var = 300;

is stored on the stack.

The stack region appears as:

[stack]

in `/proc/<PID>/maps`.

RESULT

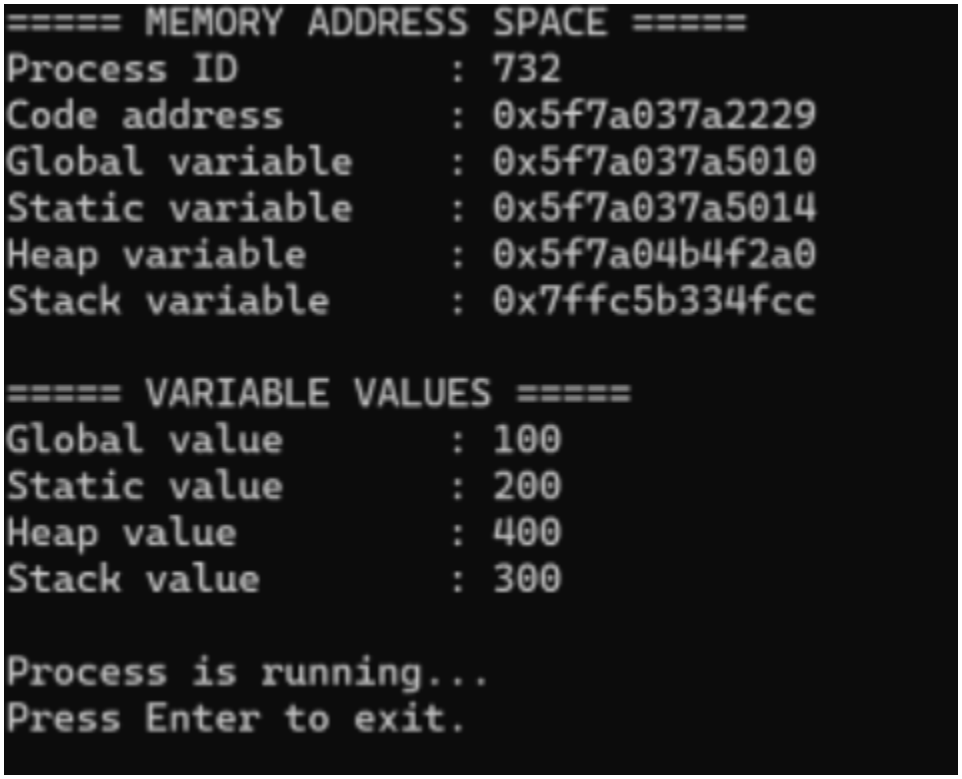
Thus, the addresses of the **code, global, static, heap, and stack variables** were successfully obtained. The Linux process address space was analyzed using the `/proc/<PID>/maps` file and memory analysis tools, and the virtual memory organization of the process was studied successfully.

```
vaishali@LAPTOP-QQD0F0TI:~$ ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
  File: original.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 12213       Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:37:05.452444785 +0000
 Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48    Inode: 12242       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
 Birth: 2026-10-30 14:37:57.009792289 +0000
```

```
73e21d240000-73e21d243000 rw-p 00000000 00:00 0
73e21d249000-73e21d24b000 rw-p 00000000 00:00 0
73e21d24b000-73e21d24f000 r--p 00000000 00:00 0
          [vvar]
73e21d24f000-73e21d251000 r--p 00000000 00:00 0
          [vvar_vclock]
73e21d251000-73e21d253000 r-xp 00000000 00:00 0
          [vdso]
73e21d253000-73e21d254000 r--p 00000000 08:30 18224
          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73e21d254000-73e21d27f000 r-xp 00001000 08:30 18224
          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73e21d27f000-73e21d289000 r--p 0002c000 08:30 18224
          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73e21d289000-73e21d28b000 r--p 00036000 08:30 18224
          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73e21d28b000-73e21d28d000 rw-p 00038000 08:30 18224
          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffc5b316000-7ffc5b337000 rw-p 00000000 00:00 0
          [stack]
```

```
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48    Inode: 12242       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
 Birth: 2026-10-30 14:37:57.009792289 +0000
vaishali@LAPTOP-QQD0F0TI:~$ find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
./symlink.txt
./hardlink.txt
./original.txt
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux World" > mmapfile.txt
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
vaishali@LAPTOP-QQD0F0TI:~$ nano mmap_program.c
vaishali@LAPTOP-QQD0F0TI:~$ gcc mmap_program.c -o mmap_program
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
vaishali@LAPTOP-QQD0F0TI:~$ ./mmap_program
-bash: ./mmap_program: No such file or directory
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
```

```
732:   ./memory_layout
Address           Kbytes     RSS   Dirty Mode  Mapping
00005f7a037a1000       4       4       0 r---- memory_layout
00005f7a037a2000       4       4       0 r-x-- memory_layout
00005f7a037a3000       4       4       0 r---- memory_layout
00005f7a037a4000       4       4       4 r---- memory_layout
00005f7a037a5000       4       4       4 rw--- memory_layout
00005f7a04b4f000     132       4       4 rw---   [ anon ]
000073e21d000000     160     160       0 r---- libc.so.6
000073e21d028000    1572     992       0 r-x-- libc.so.6
000073e21d1b1000     316     128       0 r---- libc.so.6
000073e21d200000      16      16      16 r---- libc.so.6
000073e21d204000       8       8       8 rw--- libc.so.6
000073e21d206000      52      20      20 rw---   [ anon ]
000073e21d240000      12       8       8 rw---   [ anon ]
000073e21d249000       8       4       4 rw---   [ anon ]
000073e21d24b000      16       0       0 r----   [ anon ]
000073e21d24f000       8       0       0 r----   [ anon ]
000073e21d251000       8       4       0 r-x--   [ anon ]
000073e21d253000       4       4       0 r---- ld-linux-x86-64.s
o.2
000073e21d254000     172     172       0 r-x-- ld-linux-x86-64.s
o.2
000073e21d27f000      40      40       0 r---- ld-linux-x86-64.s
o.2
000073e21d289000       8       8       8 r---- ld-linux-x86-64.s
```


===== MEMORY ADDRESS SPACE =====
Process ID : 732
Code address : 0x5f7a037a2229
Global variable : 0x5f7a037a5010
Static variable : 0x5f7a037a5014
Heap variable : 0x5f7a04b4f2a0
Stack variable : 0x7ffc5b334fcc
===== VARIABLE VALUES =====
Global value : 100
Static value : 200
Heap value : 400
Stack value : 300
Process is running...
Press Enter to exit.